Flowbit AI Architecture

Arc AI · Meetings AI (notetaker bot, transcription, minutes) · Model selection & credits

Version 1.0 · 16 Oct 2026 · For the Flowbit engineering team

What is in this document

1. The big picture: every AI piece and how they connect
2. Arc AI: current architecture and user flow
3. Meetings AI: architecture, the notetaker bot and upload flows
4. Meetings AI: background jobs, webhooks, limits and costs
5. Proposed: multi-model AI gateway (Claude, Gemini, GPT, own model later)
6. Proposed: credits, wallet and billing
7. Build phases, decisions summary, env variables and file map
8. Decisions: what we do and what we don't

How to read the labels

BUILT Live in the code today

PROPOSED Agreed direction, not built yet

DECIDE Needs a decision before building

DECIDED Agreed decision (see page 11)

Arrows in the diagrams

Request / data flow

Fallback, failure or retry path

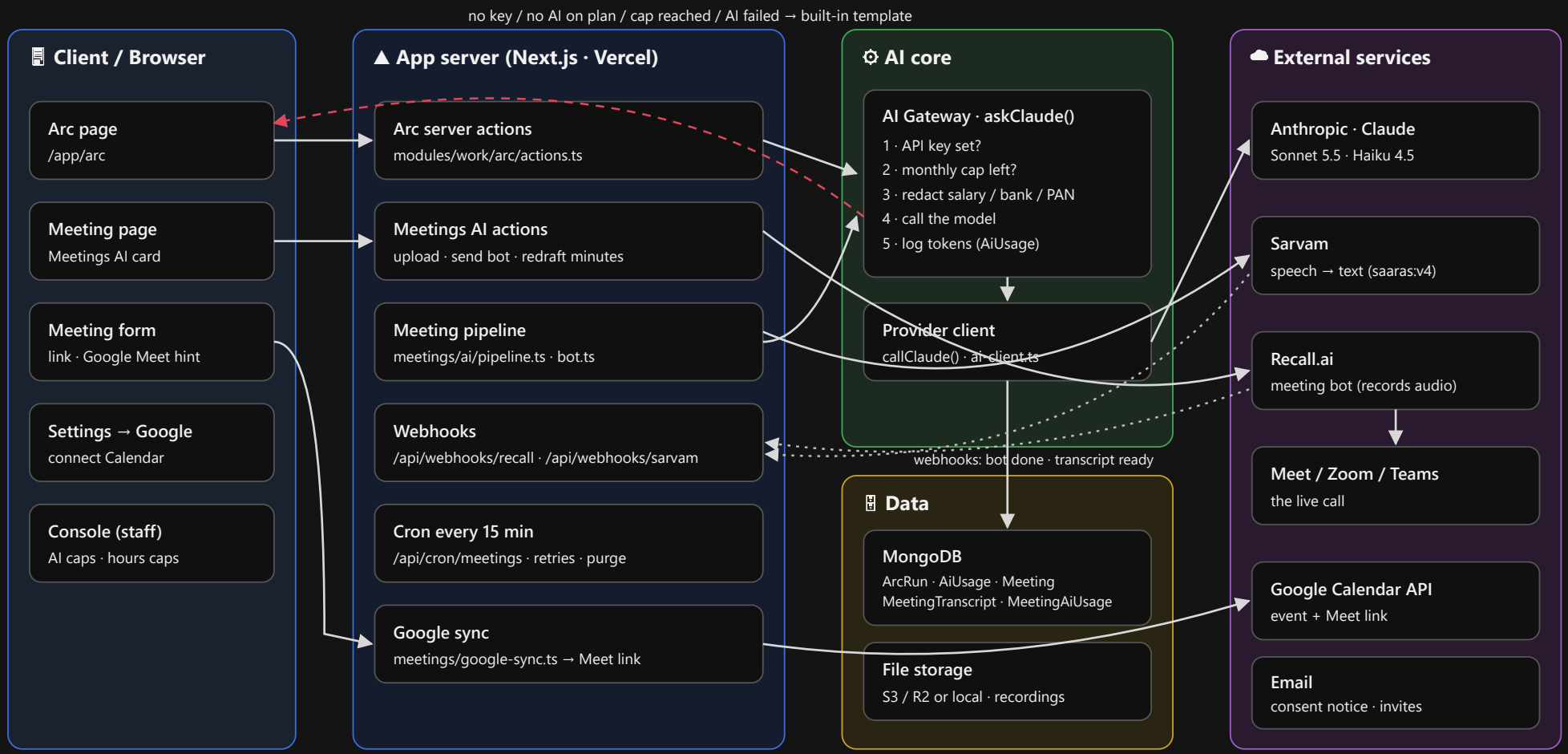
Webhook / callback (async)

Future / optional

Words used in this document

Term	Meaning
Arc AI	The assistant page `/app/arc` : drafts project outlines, task breakdowns and progress updates.
Meetings AI	Recording → transcript → minutes of meeting (MOM) + action items. A paid add-on (`meetings_ai`).
Notetaker / bot	"Flowbit Notetaker", a bot run by **Recall.ai** that joins a Meet, Zoom or Teams call and records the audio.
Sarvam	Speech-to-text service (model `saaras:v4` , Hinglish code-mix, speaker labels). Turns audio into a transcript.
AI Gateway	The one function every AI request goes through (today `askClaude()`): key check, monthly cap, redaction, usage log.
Token / Credit	Token = the provider's billing unit (~¾ of a word). Credit = Flowbit's own unit that customers see and buy (proposed).

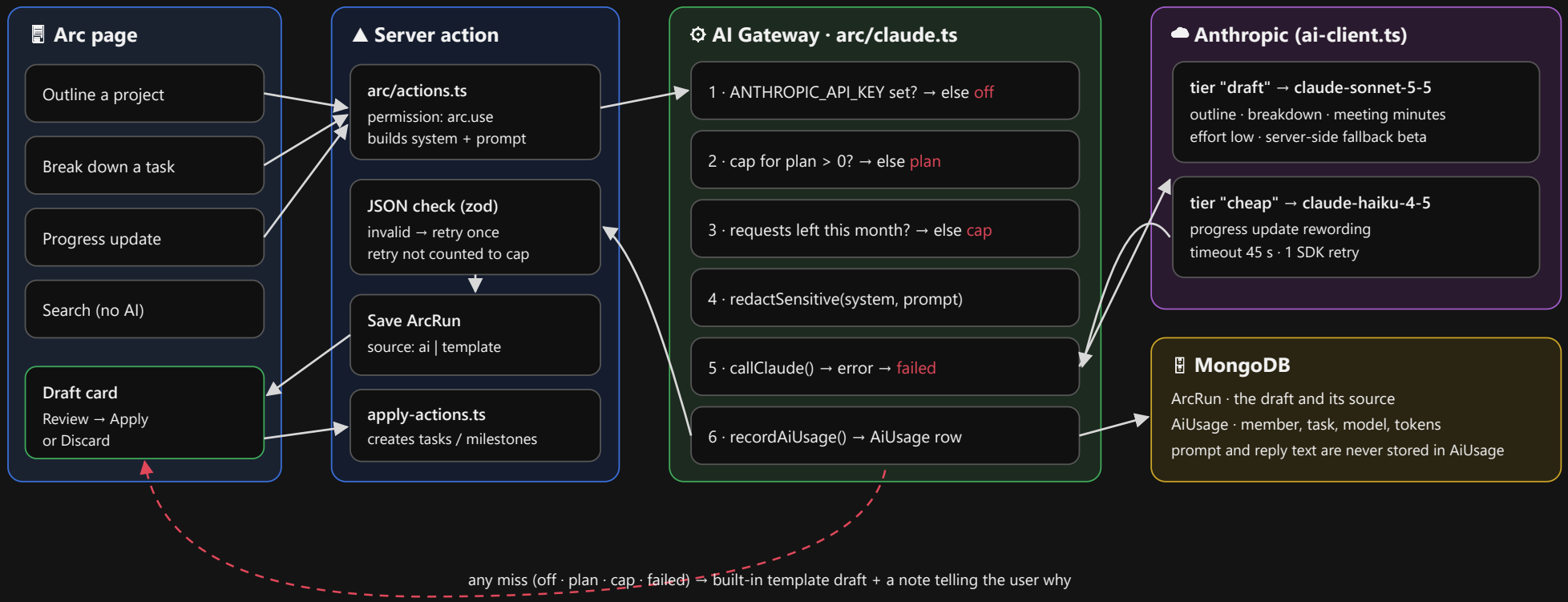
⌐ 1 · The big picture BUILT



One gateway for all AI: Arc and Meetings minutes both call the same `askClaude()`, so caps, redaction and usage logging apply everywhere. Only `ai-client.ts` talks to Anthropic.

Three outside services for meetings: Recall.ai records, Sarvam transcribes, Claude writes the minutes. Recall and Sarvam call back through webhooks; the 15-minute cron catches anything missed.

⌐ 2 · Arc AI: architecture BUILT



Monthly caps (Console)

Requests per workspace per month (IST): **free 0**, **trial 50**, **paid 500**, comped / internal 500, test 50. Staff change them in Console → Settings → Arc AI.

Redaction

Salary, bank and PAN details are stripped from the system prompt and the user prompt before anything leaves Flowbit (`lib/ai-redact.ts`).

Never blocks the user

The gateway never throws. If AI is unavailable the user still gets a template draft (`templates.ts`) and a note explaining why.

⌐ 2 · Arc AI: user flow in detail BUILT

Example: "Outline a project"

1. User opens **Arc** (`/app/arc`) and clicks **Outline a project**. A dialog asks for the project name, goal and start date (or an existing project).
2. The browser calls the server action `draftOutline`. It checks the `arc.use` permission and, for an existing project, that the user may open it.
3. The action builds the prompt: project name, goal, workspace team names, and the exact JSON shape wanted (3–6 milestones, 2–5 tasks each).
4. `askClaudeJson()` → `askClaude()`: key check → plan cap → monthly count → redaction → `callClaude()` on the **draft** tier (Claude Sonnet 5.5).
5. The reply is parsed as JSON and checked with zod. If it is invalid, Arc asks once more with "reply with only the JSON". The retry is logged but not counted against the cap.
6. Tokens are logged in `AiUsage`. The draft is saved as an `ArcRun` (status *draft*, source *ai* or *template*).
7. The page refreshes and shows the **draft card**. Nothing is created yet. The user reviews and edits it.
8. **Apply** → `apply-actions.ts` creates the project, milestones and tasks. **Discard** → the run is marked discarded and nothing is created.

The four Arc actions

Action	AI?	Tier / model	Output
Outline a project	Yes	draft · Sonnet 5.5	Milestones + tasks (JSON), Apply creates them
Break down a task	Yes	draft · Sonnet 5.5	3–7 subtasks, Apply adds them (same edit rule as tasks)
Progress update	Yes	cheap · Haiku 4.5	Facts gathered by code, AI only rewords them (max 180 words)
Search / task key	No	none	Workspace search results, or jump to the task

When the template is used

Reason	Cause	Note shown to user
`off`	No ANTHROPIC_API_KEY on the server	(none, template only)
`plan`	Cap is 0 (Free plan)	"AI drafting is on paid plans and trials…"
`cap`	This month's requests used up	"…resets on the 1st."
`failed`	API error, refusal, bad JSON twice	"…the AI service didn't respond."

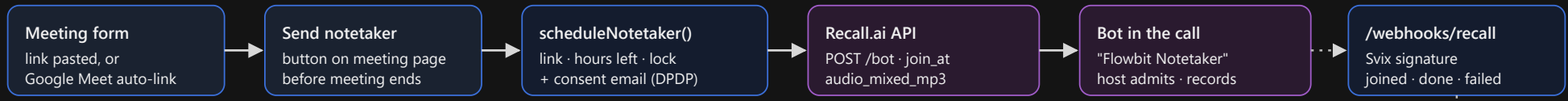
> **Design rule:** facts come from Flowbit's database, the AI only writes or rewords. For the progress update the numbers, names and titles are fixed by code and the prompt forbids adding facts.

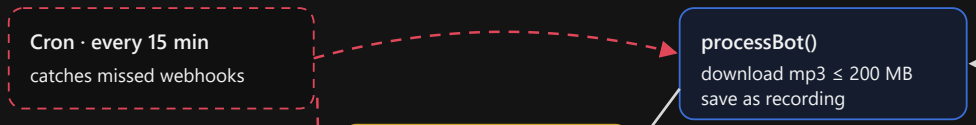
Key files

`modules/work/arc/actions.ts` actions · `claude.ts` gateway · `ai-client.ts` Anthropic SDK · `usage.ts` caps + logging · `schemas.ts` JSON shapes · `templates.ts` fallbacks · `apply-actions.ts` Apply · `lib/ai-caps.ts` · `lib/ai-redact.ts` · `models/work/AiUsage.ts`

⌐ 3 · Meetings AI: architecture BUILT

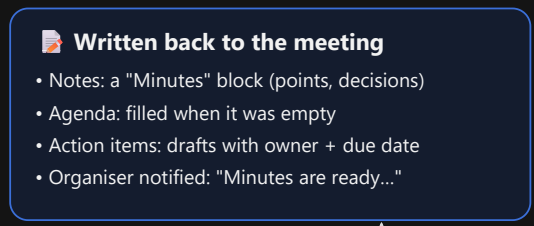
A · Notetaker bot path





B · Upload path







Which AI model gets the transcript? Today the transcript goes to **Claude Sonnet 5.5** (`claude-sonnet-5-5`, the "draft" tier) through the same AI Gateway as Arc, max 4,000 output tokens, and it counts as one Arc AI request. Decided for the new design: it goes to the organiser's default model, then the workspace default, then Claude Haiku (pages 8 and 11). Sarvam (transcription) and Recall (bot) do not change.

⌐ 3 · Meetings AI: user flows in detail BUILT

Flow A · The bot joins the meeting

1. **Create the meeting** (title, time IST, attendees, link). Paste a Meet / Zoom / Teams link, or leave it empty. If the organiser has connected Google Calendar, saving creates a **Google Meet link** automatically (the form says so; if not connected it shows a Connect button).
2. On the meeting page, the Meetings AI card shows **Send notetaker** (needs: link, meeting not ended, user is a participant, Meetings AI on the plan, Recall keys on the server). User confirms.
3. Server checks: link is Meet/Zoom/Teams, enough **hours left** for the scheduled length, and locks the meeting so two clicks can't book two bots.
4. Flowbit asks Recall.ai to book a bot with the meeting link. Booked 10+ minutes ahead → it joins at the start time; later → it joins now. Every attendee gets a **consent email** (who records, why, how long it is kept). Status: *Booked*. "Call off notetaker" cancels it.
5. At start time the bot opens the link as a guest named **Flowbit Notetaker**. The host must **admit** it (it waits up to 10 min, and leaves if nobody joins in 10 min). It posts a chat message that it is recording. Status: *In the call*.
6. Meeting ends → Recall sends a **done** webhook → Flowbit downloads the mp3, saves it as the meeting's recording and starts transcription. Backups: **Fetch from bot** button and the cron (after 20 min, up to 5 tries).
7. From here it is the **shared pipeline** (steps 8–11 on the right).

Flow B · The user records it themselves

1. User records the meeting another way: Meet/Zoom's own record button, a phone recorder for an in-person meeting, any tool. Up to **2 hours, 200 MB**.
2. On the meeting page → **Upload recording**. The browser reads the length; the server checks the hours left this month.
3. With S3/R2 storage the file goes straight to the bucket (signed PUT); otherwise through the server. Then transcription starts.

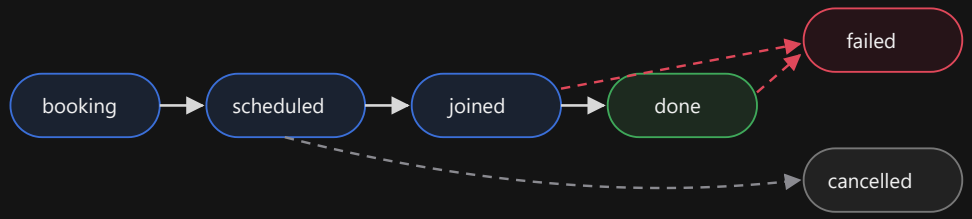
Shared pipeline (both flows)

8. **Sarvam** transcribes (Hinglish code-mix, speaker labels). It calls `/api/webhooks/sarvam` with a secret token; Flowbit then fetches the transcript. Backups: **Check now** button and the cron.
9. The transcript is saved and the **audio hours are counted** against the monthly limit.
10. **Claude** gets the transcript + title, date, attendees, agenda and returns JSON: agenda, notes, decisions, action items (owner as spoken, due date only if said). Owners are matched to workspace members.
11. Minutes go into the notes, the agenda (if empty) and draft action items. The organiser is notified. Later: rename speakers, **Draft minutes again**, delete recording/transcript; both auto-delete after the retention period (default **90 days**).

> If AI is off or the cap is reached, the notes quote the transcript and say why. Upload sends no consent email (the user recorded it themselves).

⌐ 4 · Meetings AI: background jobs, limits & costs BUILT

Status of a notetaker



Cron `/api/cron/meetings` (every 15 minutes)

Job	Picks up	Retries
Poll stuck transcripts	Sarvam jobs still "processing" after 5 min (callback missed)	each run
Bot pick-up	Meetings that ended 20+ min ago whose recording was not fetched	up to 5
Missing minutes	Transcripts done 10+ min ago without minutes (last 7 days)	up to 3
Retention purge	Recordings/transcripts older than the workspace's days (dry run unless `MEETINGS_PURGE`)	each run

Security of webhooks

Recall: Svix signature with `RECALL_WEBHOOK_SECRET`; the bot id, workspace and meeting must all match. Sarvam: a one-time token per job, only its hash is stored. Both reply 200 fast and do the work afterwards (`after()`).

Limits

Limit	Value	Where
Meetings AI hours / month	Free 0 · Trial 1 h · Paid 5 h per seat	`lib/meeting-caps.ts`, Console
Arc AI requests / month (also used by minutes)	Free 0 · Trial 50 · Paid 500	`lib/ai-caps.ts`, Console
Recording length / size	2 hours · 200 MB	`pipeline.ts`, `actions.ts`
Retention	Default 90 days (7 – 3650)	Settings → Privacy
Bot waiting room / nobody joined	10 min each, then it leaves	`recall.ts`

What one meeting costs Flowbit

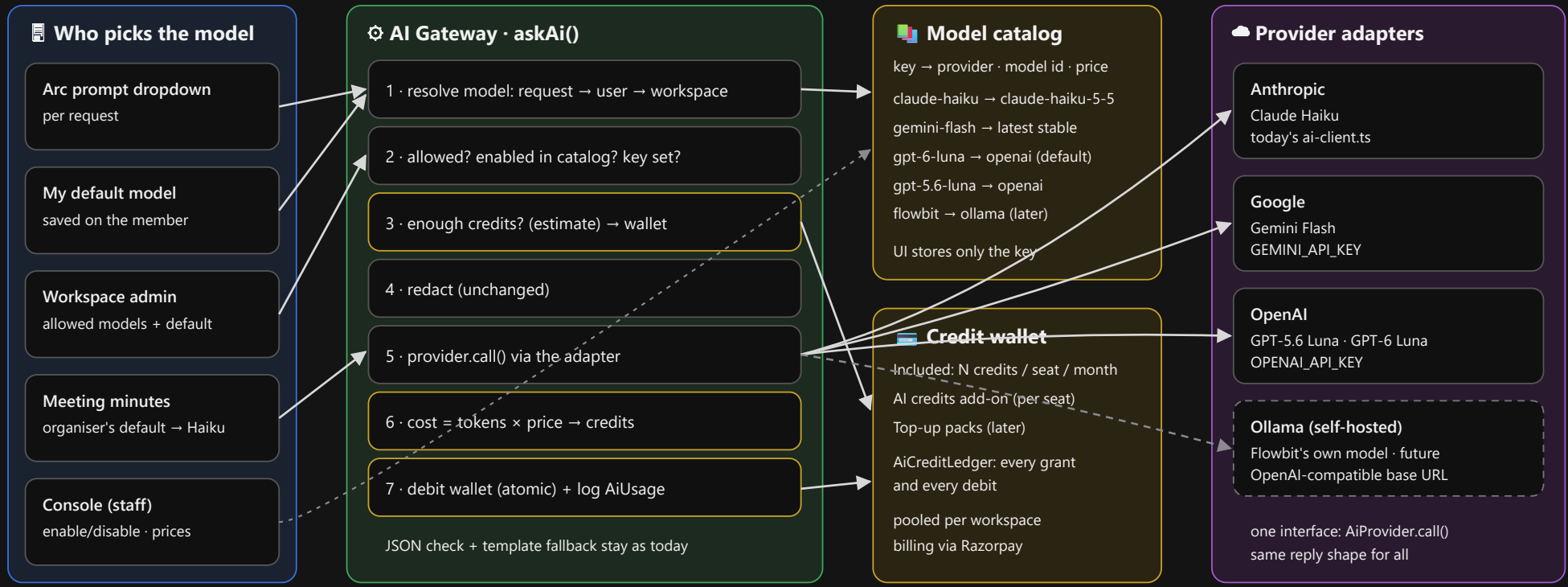
Step	Paid to	Charged by	Bot	Upload
Record the call	Recall.ai	~$0.50 / recording hour (Oct 2026 pay-as-you-go)	✓	–
Speech → text	Sarvam	per hour of audio	✓	✓
Minutes	Anthropic (later: chosen model)	per token, a few cents	✓	✓

> **Pricing rule:** the per-seat Meetings AI add-on must cover Recall + Sarvam. The hours limit counts transcribed audio, which is about the same length the bot records.

Bot options (provider behind `MeetingBotProvider`)

Keep Recall.ai now (built, reliable, no monthly fee). Cheaper APIs (MeetingBaaS, Skribby) = one new adapter. Self-hosted bots: not worth it yet. **Future free path:** import recordings from Google Meet / Zoom APIs.

⌐ 5 · Proposed: multi-model AI gateway PROPOSED



Why a catalog key

The user picks `gemini-flash`, never a raw model id. Upgrading the model behind it changes one catalog line, not every user's setting. Staff can switch a model off during an outage.

Why adapters

Same idea as `TranscriptionProvider` (Sarvam) and `MeetingBotProvider` (Recall) that already exist. Adding our own Ollama model later = one adapter + one catalog line.

What changes for features

Arc and minutes stop passing a tier and pass the resolved model. `askClaude` → `askAi`. JSON checks, redaction and templates stay. Transcription (Sarvam) and the bot (Recall) are not affected.

⌐ 6 · Proposed: credits, wallet & billing PROPOSED

Tokens inside, credits outside

Providers bill Flowbit in **tokens**, and each model has its own price per token (input and output priced differently). Customers see and buy **credits**, Flowbit's own unit. The gateway converts tokens to credits using each model's price, so cheaper models use fewer credits and Flowbit never sells below cost.



Example with made-up prices: Model A ($1 / $5 per 1M tokens) → $0.007 → 7 credits.
Model B ($0.50 / $3 per 1M tokens) → $0.004 → 4 credits. Same 3,000 tokens, different cost.
The dropdown can show "about 7 credits per draft" so users see the trade-off.

Where credits come from

Source	How	Expires
Included	With the plan: free 0, trial flat, paid N per seat (same formula as meeting hours)	Monthly reset (IST)
AI credits add-on	Paid per seat, next to `meetings_ai` in `ADD_ONS`, on the Razorpay subscription	Monthly
Top-up packs (later)	One-time purchase when a workspace runs out mid-month	12 months

Spent in order: included → add-on → top-up (soonest to expire first). Pooled for the whole workspace (seats × N), like Meetings AI hours.

Data changes

Collection	Change
`AiUsage`	+ `provider`, `modelKey`, `costUsd` (what Flowbit paid), `credits` (what we charged) → margin reports for free
`AiCreditLedger` (new)	One row per grant (monthly, add-on, top-up) and per debit. Audit trail for disputes.
Member / Org settings	Default model per member; allowed models + default per workspace
Console settings	Catalog overrides: enabled, prices, credit value, USD→INR rate

Rules for safe charging

- **Check before, charge after:** estimate from max tokens, call, then debit the real amount. A tiny overdraft is acceptable.
- **Atomic debit** (`$inc` with a condition) so two requests at once can't both spend the last credits.
- **JSON retries:** Flowbit absorbs them (the model's fault); failed calls and refusals are never charged.
- **Outage:** template fallback + a "Try with another model" button, never a silent switch.
- **Self-hosted model:** no per-token bill, so give it an internal price (GPU cost) and credits still work.

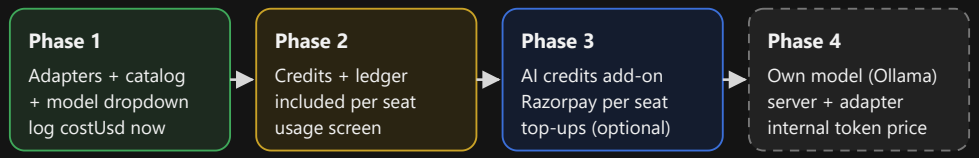
What does NOT use credits

Meeting **transcription** (Sarvam) and the **bot** (Recall) stay on the **hours** limit. Only the minutes step (text AI) spends credits. So a bot-recorded meeting uses hours + credits.

> **Privacy:** adding Google and OpenAI means listing them as subprocessors (`modules/marketing/legal/providers.ts`) and updating the privacy policy. Redaction runs for every provider.

⌐ 7 · Build phases, decisions, config & file map

Build phases



Decisions summary DECIDED details on page 11

#	Question	Decision
1	Exact model ids	`claude-haiku-5-5` · latest stable Gemini Flash · `gpt-6-luna` (default) + `gpt-5.6-luna`
2	Which model writes meeting minutes	Organiser's default → workspace default → Claude Haiku; dropdown on "Draft minutes again"
3	Value of 1 credit; included credits per seat	Round rupee value; price at 2–3× real cost; numbers set after 2–4 weeks of `costUsd` data
4	Pool per workspace or per user	Workspace pool; optional per-member limit later
5	Charge for JSON retries	No, Flowbit absorbs (max 1 retry)
6	Provider outage behaviour	Template + "Try with another model"; never a silent switch
7	Do credits expire	Included + add-on: monthly. Top-ups: 12 months

Environment variables

Variable	For	
`ANTHROPIC_API_KEY`	Claude (Arc + minutes)	NOW
`SARVAM_API_KEY`	Transcription	NOW
`RECALL_API_KEY`, `RECALL_REGION`, `RECALL_WEBHOOK_SECRET`	Notetaker bot + webhook `{APP_URL}/api/webhooks/recall`	NOW
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Google Calendar connect + Meet links (OAuth, scope `calendar.events`)	NOW
`APP_ENCRYPTION_KEY`	32-byte key that seals Google tokens	NOW
`APP_URL`	Builds callback / webhook URLs (must match Google redirect URIs)	NOW
`STORAGE_DRIVER` (+ S3/R2 keys)	Recordings storage, direct uploads	NOW
`MEETINGS_PURGE`	Turns the retention purge from dry run to real	NOW
`GEMINI_API_KEY`, `OPENAI_API_KEY`	New adapters	PLAN
`OLLAMA_BASE_URL`	Own model, later	PLAN

File map

Area	Files
Arc AI	`src/modules/work/arc/*` · `src/components/work/arc/*` · `src/app/app/arc/page.tsx`
AI caps / redaction / usage	`src/lib/ai-caps.ts` · `src/lib/ai-redact.ts` · `src/models/work/AiUsage.ts`
Meetings AI	`src/modules/work/meetings/ai/*` (pipeline, bot, recall, sarvam, minutes, job, purge)
Webhooks / cron	`src/app/api/webhooks/recall` · `/sarvam` · `src/app/api/cron/meetings`
Google / Meet link	`src/modules/google/*` · `meetings/google-sync.ts` · `api/integrations/google/*`
UI	`meeting-ai-card.tsx` · `meeting-form-dialog.tsx`

⌐ 8 · Decisions: what we do and what we don't DECIDED

#	Topic	✅ Do	❌ Don't
1	**Model ids**	Claude Haiku → `claude-haiku-5-5` (also move Arc's progress update off `claude-haiku-4-5`). Gemini Flash → latest **stable** id, copied from Google's model page. OpenAI → `gpt-6-luna` as default + `gpt-5.6-luna`; hide the unused one once data exists.	Use preview / experimental versions. Show raw model ids to users (show "Gemini Flash").
2	**Minutes model**	Organiser's default model → workspace default → Claude Haiku. Model dropdown on "Draft minutes again". Show "this meeting used X credits" on the meeting page.	Keep minutes hard-wired to Sonnet 5.5 (bigger and pricier than the catalog models, and the user can't control it).
3	**Credit value & included credits**	Build Phase 1 first and log real `costUsd` for 2–4 weeks, then set numbers. 1 credit = a round rupee value (e.g. ₹0.10 of customer price). Selling price 2–3× real provider cost. Size included credits as "about N drafts per seat per month". Show an estimate in the dropdown ("about 3 credits").	Guess numbers before there is data. Sell at cost. Show tokens to customers.
4	**Pool**	One pool per workspace (seats × credits per seat), like Meetings AI hours. Later: optional per-member monthly limit set by admins. Warn admins at 80% and 100%.	Separate wallets per user (more code; light users' credits sit unused while heavy users run out).
5	**JSON retries**	Flowbit absorbs the cost; keep the max of one retry; still log retry cost to spot models that fail often.	Charge for retries, failed calls or refusals. Charge only when the user gets an AI result.
6	**Provider outage**	Template draft + a "Try with another model" button. Console switch to disable a model everywhere. Minutes: template, cron retries (up to 3), then "Draft minutes again" with another model.	Silently switch providers (the user chose the model; another may cost more or send data to a company they didn't pick).
7	**Expiry**	Included credits: monthly reset, no rollover. Add-on credits: monthly. Top-ups: 12 months from purchase. Spend order: included → add-on → top-up.	Make credits never expire (unused prepaid credits stay on the books forever).

General rules

✅ Do	❌ Don't
Log the real cost of every request from day one (Phase 1)	Launch paid credits before there is cost data
Test each new provider with a small internal group before releasing it to all customers	Let customers bring their own API keys yet (complicates billing and support)
Add Google and OpenAI to the subprocessor list and privacy policy before launch	Send data to new providers before the legal pages are updated
Keep redaction and JSON validation for every provider	Assume every model follows the JSON format equally well; test each one
Build our own Ollama model last, once usage volumes are known	Start the self-hosted model now (GPU servers are expensive if barely used)

> **In one line:** workspace credit pool · minutes use the organiser's model (fallback Haiku) · Flowbit absorbs retries · no silent provider switching · top-ups expire after 12 months · set credit prices only after 2–4 weeks of real cost data.